Cluster lifecycle and HA

Design a highly available control plane, bootstrap with kubeadm, manage certificates, upgrade safely, replace nodes, back up etcd and rehearse recovery.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Designing a highly available control plane (12 min)
2. Bootstrapping with kubeadm: configuration and phases (12 min)
3. Certificates: PKI, expiry and rotation (12 min)
4. Version skew and upgrade strategy (12 min)
5. Node lifecycle: joining, draining and replacing (12 min)
6. Backup and disaster recovery for the cluster (13 min)
7. Cluster API and declarative cluster management (12 min)
8. Capstone: plan and rehearse an upgrade and a recovery (14 min)
Cheat sheet

Lesson 1. Designing a highly available control plane

12 minutes. Choose between stacked and external etcd topologies, place a load balancer in front of the API servers, and plan failure domains for a highly available cluster.

A single control plane node is a single point of failure for every change in the cluster. High availability means running several control plane nodes so that the loss of one does not stop operations. The design choices are few but each has consequences for cost, failure tolerance and recovery.

What you will learn

- Stacked and external etcd topologies
- Why three control plane nodes is the common minimum
- The load balancer and the control plane endpoint
- Failure domains and zone placement
- What high availability does not protect

Stacked or external etcd

In a stacked topology each control plane node runs etcd as a static pod next to the API server, scheduler and controller manager. It needs fewer machines and is simple to operate with kubeadm. The cost is coupling: losing a node loses an API server and an etcd member together. In an external topology etcd runs on its own nodes. It separates failure domains and lets you size and secure etcd independently, at the price of more machines to manage.

Why three

Raft needs a majority, so one member gives no tolerance, two gives none either, and three tolerates one failure. Three control plane nodes is therefore the usual minimum for production. Five tolerates two failures and suits very large or critical clusters, but adds write latency because more members must acknowledge.

The control plane endpoint

Nodes and users must reach any healthy API server through one stable address. A load balancer or a virtual IP in front of the API servers provides it, and it is given to kubeadm as the control plane endpoint. Health checks should test the API server port or its /readyz, and the load balancer itself must not become a single point of failure. Kubeconfig files and kubelets use this endpoint, so it is hard to change later.

Failure domains

Spread control plane nodes across failure domains such as availability zones or racks, so that one zone failing leaves a majority. With three nodes put one in each of three zones. Two zones are worse than they look, because losing the zone that holds two members loses quorum. Keep network latency between members low, since etcd commits depend on it.

What HA does not cover

Several control plane nodes do not protect against a bad upgrade applied to all of them, a mistaken delete, corrupted data replicated to every member, or loss of the whole site. Those need tested backups and a recovery plan, covered later in this module. Workers are separate: run enough of them in several zones.

```
kubeadm init --control-plane-endpoint "k8s-api.example.internal:6443" --upload-certs --pod-network \
    -cidr=10.244.0.0/16
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Designing a highly available control plane

Part	What it does	What goes wrong
Load balancer	One stable API endpoint.	Single instance.
Control plane A	API, etcd member.	Zone outage.
Control plane B	API, etcd member.	Zone outage.
Control plane C	API, etcd member.	Zone outage.
Workers	Run workloads.	Too few zones.

Flow: Load balancer to Control plane A / Load balancer to Control plane B / Load balancer to Control plane C / Control plane B to Workers

Incident: Two zones, one of them lost

Incident: *A cluster with three control plane nodes in two zones lost the zone that held two of them.*

```
$ kubectl get nodes
Unable to connect to the server: net/http: TLS handshake timeout
$ etcdctl endpoint health --endpoints=https://10.0.1.5:2379
https://10.0.1.5:2379 is unhealthy: failed to commit proposal: context deadline exceeded
(this is a simulated illustration, not a real incident)
```

Two of three etcd members were in the zone that failed, so quorum was lost and the remaining API server could not serve requests. Placing one node in each of three zones would have left a majority. The team restored quorum by recovering a lost member, then moved a node to a third zone. Count members per failure domain, not only nodes in total.

Common mistakes

- Using two control plane nodes and calling it highly available
- Putting two members in the same failure domain of three
- Using a single load balancer instance
- Choosing the control plane endpoint casually
- Treating HA as a replacement for backups

Check yourself

How many control plane nodes is a common production minimum?

Answer: Three

What is a drawback of stacked etcd?

Answer: Losing a node loses an API server and an etcd member together

Why do kubelets use a load balancer endpoint?

Answer: So they reach any healthy API server through one stable address

What does HA not protect against?

Answer: A bad change replicated to all control plane nodes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Bootstrapping with kubeadm: configuration and phases

12 minutes. Use kubeadm configuration files and phases to create reproducible clusters, join nodes and troubleshoot a bootstrap that fails part way.

kubeadm sets up the pieces of a cluster in a defined order and leaves the nodes in a known state. Using a configuration file and understanding the phases means a cluster can be rebuilt the same way each time and a failed bootstrap can be repaired instead of restarted blindly.

What you will learn

- Prerequisites on every node
- The main phases of kubeadm init
- A configuration file instead of flags
- Joining control plane and worker nodes
- Resetting and retrying safely

Prerequisites

Every node needs a container runtime with the CRI enabled, the kubelet, kubeadm and kubectl packages at matching versions, swap disabled unless configured for it, required kernel modules and sysctls for bridging and IP forwarding, unique host names, and open ports between nodes. Most bootstrap failures are a missed prerequisite, and `kubeadm init` runs preflight checks that name them.

Phases of init

`kubeadm init` runs phases in order: preflight, certificate generation, kubeconfig files, kubelet start, control plane static pod manifests, etcd static pod, wait for the control plane, upload configuration, mark the control plane node, bootstrap tokens and add-ons such as CoreDNS and kube-proxy. You can list and run a single phase with `kubeadm init phase`, which is useful for repairs, for example regenerating a certificate.

A configuration file

Flags are fine for a lab. For anything repeatable, use a configuration file with the InitConfiguration and ClusterConfiguration kinds. It records the Kubernetes version, the control plane endpoint, the pod and service subnets, certificate SANs and component flags.

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: v1.31.2
controlPlaneEndpoint: k8s-api.example.internal:6443
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
apiServer:
  certSANs:
    - k8s-api.example.internal

kubeadm init --config kubeadm.yaml --upload-certs
```

Joining nodes

Workers join with a token and the CA hash printed by init. Tokens expire after 24 hours by default, so create a new one with `kubeadm token create --print-join-command`. Additional control plane nodes join with the control plane flag and a certificate key, which is valid for two hours. After joining, install a network plugin if you have not, then check that all nodes are Ready.

MARGA by EknathaLabs
marga.eknathalabs.com

Reset and retry

A failed init or join leaves files behind. `kubeadm reset` removes what kubeadm created on that node, but not CNI configuration, iptables rules or your kubeconfig. Clean those by hand before retrying. Read the preflight and phase output first, because the line that failed is usually clear.

Diagram: Bootstrapping with kubeadm: configuration and phases

Part	What it does	What goes wrong
Preflight	Checks the node.	Swap or ports.
Certificates	Creates the CA and certs.	Wrong SANs.
Control plane	Static pod manifests.	Bad flags.
Add-ons	CoreDNS and kube-proxy.	No CNI yet.
Join nodes	Token and CA hash.	Expired token.

Flow: Preflight to Certificates / Certificates to Control plane / Control plane to Add-ons / Add-ons to Join nodes

Incident: The join token that expired

Incident: *A new worker could not join a cluster built the week before.*

```
$ sudo kubeadm join k8s-api.example.internal:6443 --token abcdef.0123456789abcdef --discovery-toke \
    n-ca-cert-hash sha256:1f2e3d
error execution phase preflight: couldn't validate the identity of the API Server: could not find  \
    a JWS signature in the cluster-info ConfigMap for token ID "abcdef"
$ kubeadm token list
(no tokens)
(this is a simulated illustration, not a real incident)
```

Bootstrap tokens expire after 24 hours unless a TTL was set. The token in the old notes was gone, so discovery failed. The fix was to print a new join command on a control plane node. Do not paste old join commands into runbooks: generate a fresh one each time.

Common mistakes

- Mismatched kubeadm, kubelet and kubectl versions
- Reusing an expired token or certificate key
- Forgetting the network plugin and wondering why nodes are NotReady
- Using kubeadm reset and expecting a clean network configuration
- Building clusters with long flag lists instead of a file

Check yourself

What does a kubeadm configuration file provide?

Answer: A repeatable record of cluster settings

How do you get a fresh worker join command?

Answer: kubeadm token create --print-join-command

What does kubeadm reset NOT clean up?

Answer: CNI configuration and iptables rules

Which phase creates the control plane static pods?

Answer: The control plane phase after certificates and kubeconfigs

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Certificates: PKI, expiry and rotation

12 minutes. Map the certificates in a kubeadm cluster, check their expiry, renew them and recover when they have already expired.

A kubeadm cluster is held together by dozens of certificates. Most are valid for a year, and when they expire silently the API server and kubelets stop trusting each other. Knowing where they live and how to renew them prevents one of the most avoidable outages.

What you will learn

- The certificate authorities and the leaf certificates they sign
- Where files live and what each is for
- Checking and renewing expiry with kubeadm
- Kubelet client and serving certificate rotation
- Recovering from expired certificates

The certificate map

kubeadm creates three authorities under `/etc/kubernetes/pki`: the cluster CA, the etcd CA and the front proxy CA. From them it signs the API server serving certificate, the API server client certificates for the kubelet and etcd, the etcd server, peer and health check certificates, and the service account signing key pair. The admin, controller manager and scheduler kubeconfig files embed client certificates too.

Lifetimes

Authority certificates are valid for ten years. Leaf certificates are valid for one year. A cluster created and never upgraded will hit expiry about a year after creation. Each `kubeadm upgrade` renews leaf certificates, which is one reason regular upgrades are healthy.

Check and renew

```
kubeadm certs check-expiration
kubeadm certs renew all
sudo systemctl restart kubelet
```

Renewing writes new files, but the control plane static pods must restart to load them. Moving the manifests out of the directory and back, or restarting the kubelet, achieves that. Repeat on every control plane node, and update your copy of `admin.conf` because it embeds a renewed client certificate.

Kubelet certificates

Each kubelet has a client certificate to talk to the API server. With `rotateCertificates` enabled it asks for a renewal through a CertificateSigningRequest before expiry, and the controller manager approves and signs it. Serving certificates can also be rotated with `serverTLSBootstrap`, but those requests need approval. Look for Pending CSRs.

```
kubectl get csr
kubectl certificate approve csr-abc12
```

When they have already expired

kubectl will fail with certificate has expired errors. On a control plane node, use the local admin kubeconfig and run `kubeadm certs renew all`. If the API server itself will not start, use the files and `crictl` as in the failure diagnosis lesson. Rotate the CA only with great care, because every client of the old CA must be updated.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Certificates: PKI, expiry and rotation

Part	What it does	What goes wrong
Cluster CA	Signs leaf certs.	Key exposure.
API server cert	Serving identity.	Missing SAN.
Kubelet client cert	Node identity.	Rotation off.
etcd certs	Peer and client.	Expired.
Kubeconfig certs	Admin and components.	Stale copy.

Flow: Cluster CA to API server cert / Cluster CA to Kubelet client cert / Cluster CA to etcd certs / Kubelet client cert to Kubeconfig certs

Incident: Expired on the anniversary

Incident: *On the first anniversary of the cluster, kubectl and the controller manager stopped working at the same hour.*

```
$ kubectl get nodes
Unable to connect to the server: x509: certificate has expired or is not yet valid: current time 2 \
    026-11-14T09:00:00Z is after 2026-11-14T08:41:12Z
$ sudo kubeadm certs check-expiration | head -4
CERTIFICATE                EXPIRES                  RESIDUAL TIME   CERTIFICATE AUTHORITY
admin.conf                 Nov 14, 2026 08:41 UTC   <invalid>
apiserver                  Nov 14, 2026 08:41 UTC   <invalid>
(this is a simulated illustration, not a real incident)
```

The cluster had never been upgraded, so no leaf certificate had been renewed in a year. Running kubeadm certs renew all on each control plane node, then restarting the control plane pods, restored access. A monitor on certificate expiry, with an alert at thirty days, would have turned an outage into a ticket.

Common mistakes

- Never checking certificate expiry
- Renewing certificates but not restarting the static pods
- Forgetting to update the admin kubeconfig copy
- Leaving kubelet CSRs Pending
- Renewing on one control plane node and not the others

Check yourself

How long are kubeadm leaf certificates valid by default?

Answer: One year

What is the command to see expiry dates?

Answer: kubeadm certs check-expiration

After renewing, what must happen for the API server to use new files?

Answer: The static pods must be restarted

What does rotateCertificates do for the kubelet?

Answer: Requests a renewed client certificate before expiry

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Version skew and upgrade strategy

12 minutes. Learn the supported version skew rules and plan upgrades one minor version at a time with checks before, during and after each step.

Kubernetes releases a minor version about every four months, and each is supported for roughly a year. Staying inside the support window means upgrading regularly, and upgrading safely means respecting the skew rules between components.

What you will learn

- Supported version skew between components
- Why skipping minor versions is not allowed
- The upgrade order
- Pre-flight checks: deprecations, add-ons, backups
- In-place and replace strategies

Skew rules

The API server is the reference. The controller manager and scheduler may be at most one minor version older than the API server. Kubelets may be up to three minor versions older in recent releases, never newer. kubectl may be one minor version newer or older. kube-proxy must match the kubelet on its node. Check the current policy for your version, since the numbers have changed over time.

One minor at a time

A cluster must move through each minor version in turn. Upgrading from 1.29 to 1.31 means 1.29 to 1.30, then 1.30 to 1.31. Patch versions can be skipped within a minor. The reason is that API conversion and storage migrations are tested only between adjacent versions.

The order

Upgrade the control plane first, one node at a time, then the workers. On the first control plane node run `kubeadm upgrade plan` and `kubeadm upgrade apply`. On other control plane nodes run `kubeadm upgrade node`. For every node, drain it, upgrade the kubelet and kubectl packages, restart the kubelet and uncordon.

```
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.31.2
kubectl drain cp2 --ignore-daemonsets --delete-emptydir-data
sudo systemctl daemon-reload && sudo systemctl restart kubelet
kubectl uncordon cp2
```

Before you start

Read the release notes for deprecations and removed APIs. Run a tool such as kubent or pluto, or check apiserver_requested_deprecated_apis, to find manifests that use APIs about to be removed. Confirm that add-ons, the CNI plugin, ingress controller and operators support the target version. Take an etcd snapshot. Check that PodDisruptionBudgets will not block draining.

In place or replace

In place upgrades update the existing nodes, as above. The replace strategy builds new nodes at the new version, joins them, drains the old ones and deletes them. Replacement is cleaner for workers, especially with autoscaling groups or Cluster API, and gives a fast rollback because the old nodes still exist until you remove them. Control plane nodes with etcd data are usually upgraded in place or replaced one at a time with care.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Version skew and upgrade strategy

Part	What it does	What goes wrong
Plan and backup	Release notes and snapshot.	Skipped.
Control plane	One node at a time.	Skew breach.
Workers	Drain, upgrade, uncordon.	PDB blocks.
Add-ons	Match versions.	Incompatible CNI.
Verify	Nodes and workloads.	No smoke test.

Flow: Plan and backup to Control plane / Control plane to Workers / Workers to Add-ons / Add-ons to Verify

Incident: A removed API found after the upgrade

Incident: *After an upgrade a deployment pipeline began failing on every apply.*

```
$ kubectl apply -f ingress.yaml
error: resource mapping not found for name: "web" namespace: "shop" from "ingress.yaml": no matche \
    s for kind "Ingress" in version "extensions/v1beta1"
$ kubectl api-versions | grep networking
networking.k8s.io/v1
(this is a simulated illustration, not a real incident)
```

The manifests still used an API version that had been removed several releases earlier, and the upgrade finally crossed the removal. Scanning manifests and live objects for deprecated APIs before the upgrade would have shown it. Fix the manifests, then add the scan to the release checklist.

Common mistakes

- Skipping a minor version
- Upgrading workers before the control plane
- Draining without checking PodDisruptionBudgets
- Upgrading without a snapshot
- Ignoring add-on compatibility

Check yourself

Which component upgrades first?

Answer: The control plane

Can you upgrade from 1.29 directly to 1.31?

Answer: No, go through 1.30

Which command shows available target versions?

Answer: kubeadm upgrade plan

Why drain a node before upgrading its kubelet?

Answer: To move its pods gracefully

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Node lifecycle: joining, draining and replacing

12 minutes. Manage nodes through their life: add, label, cordon, drain, repair and retire, and avoid the traps that make draining hang.

Nodes are cattle more than pets, but each one still needs a safe path in and out. Replacing a node well means workloads move gracefully, state is preserved and the cluster keeps its capacity throughout.

What you will learn

- Adding a node and what makes it Ready
- Cordon, drain and uncordon
- Why drains get stuck: PDBs, local storage, unmanaged pods
- Node problems and taints
- Replacing and removing a node cleanly

Joining

A new node installs the runtime and kubelet, then joins with kubeadm or a cloud provider mechanism. It becomes Ready when the kubelet reports healthy and the CNI plugin has configured networking. Add labels for pool, zone and hardware so the scheduler and your selectors can use them, and taints for dedicated pools.

Cordon and drain

Cordon marks a node unschedulable, so no new pods land there, but existing pods keep running. Drain cordons and then evicts the pods, respecting PodDisruptionBudgets and graceful termination. Uncordon makes the node schedulable again.

```
kubectl cordon worker-3
kubectl drain worker-3 --ignore-daemonsets --delete-emptydir-data --timeout=300s
kubectl uncordon worker-3
```

Why a drain hangs

Drain waits for each eviction to succeed. A PodDisruptionBudget that allows zero disruptions blocks it. Pods not managed by a controller are refused unless you use force, because they will not be recreated. Pods with local storage need `--delete-emptydir-data`. DaemonSet pods are ignored with a flag. A pod that ignores SIGTERM will wait for its grace period before being killed.

Unhealthy nodes

When a node stops heartbeating, the node controller marks it NotReady and adds the `node.kubernetes.io/unreachable` taint, and after the default toleration of five minutes pods are evicted. Stateful pods with a ReadWriteOnce volume may need extra time to detach. Decide in advance whether to wait or to delete the node object after confirming the machine is really gone.

Replace and retire

To replace a node, add the new one first so capacity does not dip, drain the old one, check workloads are healthy, then delete the node object and the machine. If you delete the machine first, the node object remains until the controller notices.

```
kubectl delete node worker-3
kubectl get pods -A -o wide --field-selector spec.nodeName=worker-3
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Node lifecycle: joining, draining and replacing

Part	What it does	What goes wrong
New node	Joins and becomes Ready.	No CNI.
Cordon	Stops new pods.	Forgotten.
Drain	Evicts pods.	PDB blocks.
Retire	Delete node and machine.	Stale node object.

Flow: New node to Cordon / Cordon to Drain / Drain to Retire

Incident: The drain that never finished

Incident: *A node drain sat for forty minutes during a maintenance window.*

```
$ kubectl drain worker-3 --ignore-daemonsets --delete-emptydir-data
evicting pod shop/cart-7d9f
error when evicting pods/"cart-7d9f" -n "shop" (will retry after 5s): Cannot evict pod as it would \
     violate the pod's disruption budget.
$ kubectl get pdb -n shop
NAME   MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS
cart   2               N/A               0
(this is a simulated illustration, not a real incident)
```

The cart deployment had two replicas and a PodDisruptionBudget requiring two, so the budget allowed no disruptions and the drain retried forever. The fix was to scale to three replicas, which allowed one disruption, or to relax the budget for the window. Review PDBs against replica counts before planned maintenance.

Common mistakes

- Draining without checking PodDisruptionBudgets
- Forcing eviction of pods that are not controller managed
- Deleting the machine before draining
- Forgetting to uncordon after maintenance
- Leaving the node object behind

Check yourself

What does cordon do?

Answer: Marks the node unschedulable without evicting pods

Why might a drain hang?

Answer: A PodDisruptionBudget allows no disruptions

Which flag handles DaemonSet pods during drain?

Answer: --ignore-daemonsets

What is the safe order to replace a node?

Answer: Add the new node, drain the old one, then delete it

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Backup and disaster recovery for the cluster

13 minutes. Back up etcd and the other state you need, set recovery objectives, test restores and plan for loss of one control plane node or the whole cluster.

High availability keeps a cluster running through a node failure. It does not help when data is deleted, corrupted or when the whole site is lost. Those cases need backups, written recovery steps and rehearsals.

What you will learn

- What state needs backing up
- Choosing RPO and RTO
- Automating etcd snapshots and storing them safely
- Restoring one member and restoring a whole cluster
- Rehearsing and measuring recovery

What to back up

The essential item is etcd, which holds every object. Also keep `/etc/kubernetes/pki` and the kubeadm configuration, since certificates and the control plane endpoint matter for a rebuild. Application data in persistent volumes needs its own backups, with volume snapshots or an application-aware tool. Manifests should live in version control so that a fresh cluster can be rebuilt from them.

RPO and RTO

The recovery point objective is how much data you can afford to lose, which sets the snapshot frequency. The recovery time objective is how long the cluster can be down, which sets how automated and rehearsed the recovery must be. An hourly snapshot gives a one hour RPO. Write both numbers down, and measure them in a rehearsal.

Automate snapshots

Run snapshots from a scheduled job on a control plane node, copy them off the node to separate storage, encrypt them because they contain Secrets, and keep several generations. Alert if a snapshot is missing or its size drops sharply.

```
ETCDCTL_API=3 etcdctl snapshot save /var/backups/etcd-$(date +%F-%H%M).db --endpoints=https://127. \
    0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt \
     --key=/etc/kubernetes/pki/etcd/server.key
etcdutl snapshot status /var/backups/etcd-latest.db -w table
```

Restore scenarios

Losing one member of a healthy cluster is routine: remove the dead member with `etcdctl member remove`, add a new one and let it sync. Losing quorum, or the whole cluster, means a restore from a snapshot. Restore the same snapshot on every member, with a new cluster token and the initial cluster list, or restore on one node and rebuild the others by joining them. After the restore, pods that were created after the snapshot will be missing and objects deleted since will have returned, so reconcile.

Rehearse

A backup that was never restored is a hope. Schedule a drill in a scratch environment that restores a recent snapshot into a new cluster, checks that namespaces and workloads appear, and records how long it took. Update the runbook with what you learned.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Backup and disaster recovery for the cluster

Part	What it does	What goes wrong
etcd	Live state.	Corruption.
Snapshot job	Scheduled saves.	Silent failure.
Off-node storage	Encrypted copies.	Same failure domain.
Restore drill	Rebuild and time it.	Never rehearsed.

Flow: etcd to Snapshot job / Snapshot job to Off-node storage / Off-node storage to Restore drill

Incident: The backups that stopped in March

Incident: *During a recovery the newest snapshot was eight months old.*

```
$ ls -lt /var/backups/etcd-*.db | head -3
-rw------- 1 root root 41943040 Mar  3 02:00 /var/backups/etcd-2026-03-03-0200.db
-rw------- 1 root root 41930752 Mar  2 02:00 /var/backups/etcd-2026-03-02-0200.db
$ crontab -l | grep etcd
(no output)
(this is a simulated illustration, not a real incident)
```

A node rebuild had removed the cron entry, and nothing alerted on missing snapshots. The recovery used the March snapshot and lost months of changes, which declarative, repository-driven manifests and runbooks partly recovered. The team added an alert on snapshot age, moved the job into a controller, and scheduled a quarterly restore drill.

Common mistakes

- Never testing a restore
- Keeping backups only on the control plane node
- Storing snapshots unencrypted
- No alert on missing backups
- Forgetting that persistent volume data is separate

Check yourself

What is the most important state to back up for the cluster itself?

Answer: etcd

What sets how often to snapshot?

Answer: The recovery point objective

Why encrypt snapshots?

Answer: They contain Secrets

What proves a backup is usable?

Answer: A successful restore drill

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Cluster API and declarative cluster management

12 minutes. See how Cluster API treats clusters, machines and control planes as Kubernetes objects, and when declarative lifecycle management beats scripts.

Scripts that build clusters tend to drift, and each cluster becomes a one-off. Cluster API takes the same approach Kubernetes uses for apps: describe the cluster you want as objects and let controllers make it so, including upgrades and replacement.

What you will learn

- The idea of a management cluster and workload clusters
- Cluster, Machine, MachineDeployment and control plane objects
- How an upgrade becomes a version bump
- Providers and what they do
- When it is worth the complexity

Management and workload clusters

Cluster API runs in a management cluster, which can be a small dedicated cluster. It creates and manages workload clusters, on clouds, on virtual machines or on bare metal, through infrastructure providers. The management cluster holds the desired state of every workload cluster, so protecting it with backups matters.

The objects

A Cluster object ties the pieces together. A control plane object, commonly a KubeadmControlPlane, describes control plane machines and their kubeadm settings. A MachineDeployment is like a Deployment for worker nodes, with a template and a replica count, and a MachineSet and Machines beneath it. Infrastructure templates describe the cloud specific machine shape.

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: MachineDeployment
metadata:
  name: pool-a
spec:
  clusterName: prod
  replicas: 5
  template:
    spec:
      clusterName: prod
      version: v1.31.2
```

Upgrades as edits

To upgrade workers, change the version in the MachineDeployment template. The controller creates new machines at that version and removes the old ones, a rolling replacement. For the control plane, change the version in the control plane object, and it replaces control plane machines one at a time, keeping etcd quorum. Scaling is changing a replica count.

Providers

Providers do the cloud specific work: bootstrap providers create the kubeadm configuration for a machine, infrastructure providers create the virtual machine or server, and control plane providers manage control plane machines. Choose providers that match your platform and check their maturity.

MARGA by EknathaLabs
marga.eknathalabs.com

Is it worth it

For one small cluster, kubeadm and a runbook are simpler. With many clusters, frequent upgrades or a platform team offering clusters as a service, the declarative approach removes drift, makes replacement routine and gives an audit trail through version control. The trade is operating a management cluster and learning its failure modes.

Diagram: Cluster API and declarative cluster management

Part	What it does	What goes wrong
Management cluster	Holds desired state.	Unbacked.
Cluster object	Describes a cluster.	Wrong version.
Control plane object	Control plane machines.	Quorum risk.
MachineDeployment	Worker machines.	Bad template.
Infrastructure provider	Creates machines.	Quota.

Flow: Management cluster to Cluster object / Cluster object to Control plane object / Cluster object to MachineDeployment / MachineDeployment to Infrastructure provider

Incident: A template change that replaced every worker

Incident: *A small edit to a machine template rolled all worker nodes at once.*

```
$ kubectl get machines -n prod
NAME            PHASE          VERSION
pool-a-abc12    Provisioning   v1.31.2
pool-a-def34    Provisioning   v1.31.2
pool-a-ghi56    Deleting       v1.30.5
(this is a simulated illustration, not a real incident)
```

A change to a machine template triggers a rolling replacement, and the rollout settings allowed too many machines to be replaced at once for a workload with strict capacity needs. The fix was to set maxUnavailable to one and add PodDisruptionBudgets. Declarative tools make big changes easy, so rollout limits matter more, not less.

Common mistakes

- Not backing up the management cluster
- Treating machines as pets and editing them by hand
- Setting rollout limits too wide
- Choosing immature providers without testing
- Using it for one small cluster where it adds little

Check yourself

Where does Cluster API run?

Answer: In a management cluster

How do you upgrade workers with Cluster API?

Answer: Change the version in the MachineDeployment template

What is a MachineDeployment like?

Answer: A Deployment for worker nodes

What does a provider do?

Answer: Handles platform specific work such as creating machines

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Capstone: plan and rehearse an upgrade and a recovery

14 minutes. Put the module together by writing an upgrade plan with checks and rollback points and a recovery runbook, then rehearsing both and timing them.

The best sign of a mature cluster is that the scary tasks are boring. This capstone turns what you have learned into two documents you can hand to a colleague, an upgrade plan and a recovery runbook, and a rehearsal that proves they work.

What you will learn

- What an upgrade plan must contain
- What a recovery runbook must contain
- How to rehearse safely and measure the result
- A readiness checklist you can reuse

The upgrade plan

Write the plan as numbered steps with an owner, an expected duration and a check after each step. Include the current and target versions and the version skew check. List preparation tasks: release notes, deprecated API scan, add-on compatibility, an etcd snapshot, PodDisruptionBudget review and a maintenance window. Describe the order, control plane first, then workers in batches that keep capacity. Define a stop point after the first control plane node where you can still roll back.

Rollback thinking

Control plane upgrades are not easily reversible once storage schema migrations have run, so the safe rollback is a restore from the snapshot taken before. Worker upgrades are reversible by replacing nodes with the previous version. Say clearly what you will do at each stop point if a check fails, and who decides.

The recovery runbook

Cover the cases you can name: one control plane node lost, etcd quorum lost, all control plane nodes lost, expired certificates and a deleted namespace. For each give symptoms, the first checks, the exact commands with placeholders you fill in at the time, expected output, and how to verify the result. Keep it where you can read it when the cluster is down.

Rehearse and measure

Build a scratch cluster that mirrors production in shape, run the upgrade plan end to end, then break it on purpose and use the runbook to recover. Time each step, note surprises and fix the documents. Repeat each quarter, and after major changes.

A readiness check in code

```
checks = {
    "etcd snapshot taken and verified": True,
    "deprecated API scan clean": True,
    "add-ons support target version": True,
    "PDBs allow one disruption": False,
    "rollback owner named": True,
}
blocked = [k for k, ok in checks.items() if not ok]
print("ready" if not blocked else "blocked by: " + ", ".join(blocked))
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Capstone: plan and rehearse an upgrade and a recovery

Part	What it does	What goes wrong
Upgrade plan	Steps, checks, stop points.	Vague steps.
Runbook	Recovery commands.	Out of date.
Rehearsal	Scratch cluster.	Not like prod.
Measured times	RTO evidence.	No timing.

Flow: Upgrade plan to Runbook / Runbook to Rehearsal / Rehearsal to Measured times

Incident: The runbook that lived on the broken cluster

Incident: *During an outage the only copy of the runbook was in a wiki hosted on the failed cluster.*

```
$ curl -sI https://wiki.corp.example/k8s-recovery | head -1
curl: (7) Failed to connect to wiki.corp.example port 443: Connection refused
$ ls ~/runbooks
(no such directory)
(this is a simulated illustration, not a real incident)
```

The documentation platform ran on the same cluster that had failed, so nobody could read the steps. The team moved the runbooks to version control, copied them to a read only share and a printed binder for the on-call, and added a check that the runbook location does not depend on the cluster. Test that you can reach your recovery documents with the cluster down.

Common mistakes

- Plans without stop points or owners
- Runbooks stored on the system they recover
- Rehearsing only the happy path
- Never timing the steps
- Not updating documents after a drill

Check yourself

Where should a recovery runbook live?

Answer: Somewhere that does not depend on the cluster

What is the safest rollback for a control plane upgrade?

Answer: Restore from the pre-upgrade snapshot

Why time the rehearsal?

Answer: To measure real recovery time against the objective

How often should you rehearse?

Answer: Regularly, such as quarterly, and after major changes

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Cluster lifecycle and HA

kubeadm commands, certificate renewal, upgrade order, node replacement and etcd backup.

kubeadm basics

Job	Command
New cluster	`kubeadm init --config kubeadm.yaml --upload-certs`
Join worker	`kubeadm token create --print-join-command`
Reset a node	`kubeadm reset` then clean CNI and iptables
Check certificates	`kubeadm certs check-expiration`
Renew certificates	`kubeadm certs renew all`

Upgrade order

Step	Command
Plan	`kubeadm upgrade plan`
First control plane	`kubeadm upgrade apply v1.xx.y`
Other control planes and workers	`kubeadm upgrade node`
Drain	`kubectl drain NODE --ignore-daemonsets --delete-emptydir-data`
Back in service	`kubectl uncordon NODE`

Backup and restore

Job	Command
Snapshot	`etcdctl snapshot save FILE`
Check	`etcdctl snapshot status FILE -w table`
Restore	`etcdctl snapshot restore FILE --data-dir DIR`

Rules of thumb

Rule	Why
One minor version at a time	Skew policy and API removals
Odd number of etcd members	Quorum
Alert on snapshot age	Silent backup failure

MARGA by EknathaLabs marga.eknathalabs.com